

Documentation technique — Conception de la base de données

EcoRide

1. Contexte

EcoRide impose l'usage conjoint d'une base **relationnelle** (MySQL) et d'une base **non relationnelle** (MongoDB). Ce document justifie les choix de modélisation retenus, en s'appuyant sur les user stories (US) du sujet, afin de pouvoir les défendre devant le jury.

2. Conventions générales

Choix	Justification
Noms de tables/colonnes en anglais	Convention usuelle en développement (Symfony, Doctrine, librairies), facilite la relecture par un jury technique et évite les problèmes d'accents/mots réservés SQL.
Système de crédits en entier (<code>INT</code>), jamais en <code>DECIMAL</code>	Le sujet (US6, US9) parle uniquement de « crédits », une unité indivisible propre à la plateforme — pas d'euros/centimes. Un <code>DECIMAL(6,2)</code> aurait suggéré à tort une monnaie fractionnable.
<code>created_at</code> / <code>updated_at</code> sur les tables mutables	Permet un audit minimal (quand une réservation a changé de statut, quand un trajet a été modifié) sans complexifier le modèle métier.
Contraintes <code>CHECK</code> sur les champs sensibles	Défense en profondeur : même si Symfony valide déjà les données, la base reste cohérente en cas d'insertion directe (script, fixture, erreur applicative).

3. Choix relatifs à MySQL (base relationnelle)

3.1 Rôles utilisateur : JSON + attribut métier séparé

`user.roles` (JSON) porte uniquement les **rôles de sécurité Symfony** (`ROLE_USER`, `ROLE_EMPLOYEE`, `ROLE_ADMIN`), conformément au standard `UserInterface::getRoles()`.

Le statut **chauffeur/passager** (US8 : « *sélectionner s'il est chauffeur, passager ou les deux* ») est porté par des booléens dédiés (`is_driver`, `is_passenger`), car c'est un attribut **métier évolutif**, pas une permission d'accès. Le mélanger aux rôles de sécurité est un anti-pattern (un firewall n'a pas à interpréter un statut métier).

Choix rejeté : une entité `ROLE` classique (proposée dans l'annexe du sujet) a été écartée — le JSON suffit et évite une jointure inutile pour une donnée aussi stable.

`roles` porte un `DEFAULT (JSON_ARRAY())` : filet de sécurité utile car le compte administrateur est créé **hors application** (US13), donc potentiellement par insertion SQL directe où ce champ pourrait être oublié.

3.2 Normalisation de la marque de véhicule

Table `brand` séparée plutôt qu'un simple `VARCHAR` libre dans `vehicle`.

Justification : évite les doublons de saisie ("Peugeot" / "peugeot" / "PEUGEOT"), garantit une liste fermée exploitable pour un `<select>` côté front, et respecte la 3FN pour une donnée à faible cardinalité et forte réutilisation.

Le sujet autorise explicitement à s'écarter de l'annexe fournie pour la partie relationnelle — ce choix est donc un raffinement volontaire, pas une obligation.

3.3 Cardinalités corrigées

Règle appliquée : la cardinalité `(x,1)` porte toujours la clé étrangère vers l'autre table. Exemple retenu pour toutes les relations 1-n du modèle :

`user (0,n) – owns – (1,1) vehicle` : un utilisateur possède 0 à n véhicules ; un véhicule appartient à un seul utilisateur → la FK `owner_id` est portée par `vehicle`.

Ce principe a été appliqué systématiquement à `vehicle`, `carpool`, `booking`, `review` et `credit_transaction`.

3.4 Fusion des champs de validation post-trajet

Les deux champs initiaux `confirmation_passager` et `validation_trajet` étaient redondants et ne géraient pas explicitement le cas « le trajet s'est mal passé » (US11).

Remplacés par un unique champ `post_ride_validation ENUM('PENDING','OK','INCIDENT')` + `validated_at` : plus lisible, un seul état possible à la fois, et l'`ENUM` MySQL empêche toute valeur invalide en base.

3.5 Contraintes d'unicité

Contrainte	US concernée	Rôle
<code>UNIQUE(passenger_id, carpool_id)</code> sur <code>booking</code>	US6	Empêche un passager de réserver deux fois le même trajet.
<code>UNIQUE(passenger_id, carpool_id)</code> sur <code>review</code>	US11	Empêche un passager de déposer plusieurs avis pour un même trajet.

3.6 Politique de suppression des clés étrangères

`ON DELETE RESTRICT` systématique sur toutes les FK vers `user` et `carpool`.

Justification : le sujet impose de conserver l'historique financier (`credit_transaction`) et l'historique des trajets (US10). La suspension d'un compte se fait via `user.is_active = false`, jamais par suppression réelle — une FK en `CASCADE` risquerait de supprimer silencieusement des données réglementaires/comptables.

3.7 Préférences rattachées à l'utilisateur, pas au véhicule

`accepts_smokers`, `accepts_pets`, `custom_preferences` sont portés par `user`, pas par `vehicle` ni `carpool`.

Justification textuelle : le sujet emploie systématiquement l'expression « **préférences du conducteur** » (US5, US8), jamais « du véhicule » ou « du trajet ». Aucune US ne suggère qu'un même chauffeur applique des règles différentes selon la voiture utilisée.

Hypothèse de conception à assumer devant le jury : les préférences sont considérées comme stables pour un chauffeur donné, indépendamment du trajet ou du véhicule.

3.8 Choix rejeté : dénormalisation dans `review`

Ajouter un `driver_id` directement dans `review` (proposé pour accélérer le calcul de note moyenne) a été écarté.

Justification : le lien `review → carpool → user` reste une jointure triviale à l'échelle du projet ; dupliquer la donnée introduit un risque d'incohérence pour un gain de performance non justifié ici.

3.9 Choix rejeté : statut `INCIDENT` sur `carpool`

Un incident est déclaré **par un passager précis** (`booking.post_ride_validation = 'INCIDENT'`), pas par le trajet dans son ensemble.

Ajouter `INCIDENT` au cycle de vie `PLANNED/STARTED/COMPLETED/CANCELLED` de `carpool` créerait une ambiguïté si plusieurs passagers ont des retours différents sur un même trajet.

La détection des trajets à traiter se fait par une requête sur `booking.post_ride_validation = 'INCIDENT'`, croisée avec MongoDB pour le détail (US12).

4. Choix relatifs à MongoDB (base non relationnelle)

4.1 Pourquoi une collection `incident` en NoSQL

Le sujet impose explicitement l'usage d'une base non relationnelle en plus de la relationnelle.

`incident` est un candidat naturel : structure simple, volumétrie faible, pas de besoin de jointures complexes, et un document autonome correspond bien à un signalement ponctuel consulté par les employés (US12).

4.2 Dénormalisation volontaire

Le document `incident` duplique `departure`, `arrival`, `departure_at`, `arrival_at` plutôt que de forcer une jointure vers MySQL à chaque consultation. Justification : cohérent avec l'esprit NoSQL (lecture rapide, document auto-suffisant) et répond directement à l'US12 (« *un descriptif du trajet* » visible immédiatement par l'employé).

4.3 Références croisées MySQL ↔ MongoDB

`carpool_id`, `booking_id`, `passenger_id`, `driver_id` sont stockés en `int` (et non en `ObjectId`) car ils référencent des clés auto-incrémentées **MySQL**. Il n'existe pas de contrainte d'intégrité référentielle technique entre les deux bases (limite assumée d'une architecture polyglotte) : la cohérence est garantie **au niveau applicatif** (Symfony), à documenter comme point de vigilance en soutenance.

5. Traçabilité US → schéma (aide-mémoire soutenance)

US	Élément du schéma concerné
US6 (participer à un covoiturage)	<code>booking</code> , contrainte unique, <code>user.credits</code>
US7 (création de compte)	<code>user.credits DEFAULT 20</code> , <code>user.roles</code>
US8 (espace utilisateur)	<code>user.is_driver/is_passenger</code> , préférences, <code>vehicle</code> , <code>brand</code>
US9 (saisir un voyage)	<code>carpool</code> , <code>credit_cost_per_passenger</code> , commission plateforme via <code>credit_transaction</code>
US10 (historique)	<code>booking.status</code> , <code>carpool.status</code> , <code>cancelled_at</code>
US11 (démarrer/arrêter + avis)	<code>carpool.status</code> , <code>booking.post_ride_validation</code> , <code>review</code>
US12 (espace employé)	<code>review.status</code> (modération), collection <code>incident</code>
US13 (espace admin)	<code>user.roles</code> (ROLE_ADMIN créé hors app), <code>user.is_active</code> (suspension), agrégations sur <code>credit_transaction</code> pour les graphiques

6. Points à assumer/expliquer si le jury questionne

Pourquoi pas de FK technique entre MySQL et MongoDB → limite structurelle d'une architecture polyglotte, cohérence gérée côté applicatif.

Pourquoi les préférences sont sur `user` et non `vehicle` → hypothèse de conception justifiée par le vocabulaire du sujet, à assumer comme choix (le sujet ne tranche pas explicitement).

Pourquoi `RESTRICT` partout plutôt que `CASCADE` → priorité donnée à la conservation de l'historique et des données financières, suspension via flag plutôt que suppression.